TP2 — Objet « gain » custom

TP

DSP temps réel sur Teensy 4.x + Audio Shield v1

Guilhem PAGES **Version 1**
guilhem.pages@univ-lemans.fr **Durée : 45 min**

? Problématique

Comment écrire son *propre* objet de traitement dans l'Audio Library, et le piloter en temps réel ? On implémente un gain contrôlé en MIDI depuis le PC, avec mute et bypass.

Objectifs:

- Écrire un objet héritant de `AudioStream` ;
- Structurer `update()` : `receiveWritable()` → traitement → `transmit()` → `release()` ;
- Traiter le signal **échantillon par échantillon** dans le budget temps réel ;
- Gérer proprement un **mute** / **bypass** et la **saturation** `int16`.

Prérequis:

- TP1 validé (passthrough) ;
- Un PC avec Chrome (ou Edge) pour le pupitre MIDI.



Remarque *Pilotage en MIDI*

Les paramètres sont contrôlables en MIDI depuis le PC, par le même câble USB que l'audio. Régler *Outils* → *USB Type* → *« Serial + MIDI + Audio »*, ouvrir `outils/pupitre_midi.html` dans Chrome, choisir la sortie « Teensy MIDI/Audio » et le preset **TP2 – Gain**. Chaque fader envoie un *Control Change* (0–127) que la Teensy convertit en valeur physique via `midi_map.h`.

CC	Paramètre	Plage	Loi
7	gain	0 – 4	linéaire
80	bypass	0 / 127	interrupteur
81	mute	0 / 127	interrupteur

La partie MIDI n'est pas à étudier ni à coder : tant que vous conservez, dans `effect_gain.h`, les méthodes d'accès aux attributs telles qu'elles sont fournies, le pilotage fonctionnera.

1 Anatomie d'un objet `AudioStream`

Tous les objets qui effectuent un traitement du signal héritent de la classe `AudioStream`, qui vous impose lors de la création d'un objet de définir le nombre de voies, ainsi que la méthode `update()`, qui réalise le calcul. C'est ensuite la bibliothèque Audio qui se chargera d'appeler la fonction `update()` lorsqu'il faudra, en fonction du patch et des données.

Attention lors de la création de nouveaux attributs, car vous êtes dans un contexte concurrent et asynchrone. La méthode `update()` est appelée sur interruption, pendant que la fonction `loop()` tourne en tâche de fond. Si un attribut peut être lu dans `update()` et modifié dans `loop()`, vous pouvez vous retrouver avec des incohérences (l'attribut est en cours de modification au moment où vous l'utilisez). Pour pallier ce problème, il faut déclarer ces attributs `volatile`.

```
1  #define NB_ENTREES 1
2
3  class AudioEffectGain : public AudioStream {
4  public:
5    AudioEffectGain() : AudioStream(NB_ENTREES, inputQueueArray) {}
6    // mes méthodes
7    virtual void update(void); // méthode de calcul
8  private:
9    audio_block_t *inputQueueArray[NB_ENTREES];
10   // mes attributs définis en "volatile" s'ils peuvent être modifiés ailleurs que dans update()
11 };
```



Attention

Règles d'or **dans `update()`** : pas d'allocation dynamique, pas d'attente bloquante, pas de `Serial` ni de lecture MIDI. Toujours `release()` les blocs reçus, même en sortie anticipée. Rester dans le budget temps (≈ 2,9 ms).

2 Cœur du traitement : `update()`

La méthode `update()` de votre objet est appelée par l'ordonnanceur dès qu'un bloc audio est disponible. Avec la configuration par défaut de la bibliothèque Audio de Teensy, un bloc contient `AUDIO_BLOCK_SAMPLES` échantillons. À 44,1 kHz et avec 128 échantillons par bloc, cela correspond à une durée de

$$\frac{128}{44100} \simeq 2{,}9 \text{ ms}.$$

La première chose à faire est donc de récupérer le bloc audio correspondant à chaque entrée de l'objet. Pour cela, on utilise `receiveReadOnly(NUMERODUCANAL)` si l'on souhaite uniquement lire les échantillons, ou `receiveWritable(NUMERODUCANAL)` si l'on souhaite modifier directement le bloc.

Ces méthodes retournent un pointeur vers un objet de type `audio_block_t`. Les échantillons sont accessibles dans le tableau `block->data`, par exemple avec `block->data[i]` pour accéder au i^{e} échantillon.

Il est important de vérifier que le pointeur retourné n'est pas nul avant d'accéder au bloc. En effet, aucun bloc n'est nécessairement disponible à chaque appel de `update()`.

Une fois le traitement terminé, le bloc doit être transmis à la sortie de l'objet avec `transmit(block)` (ou `transmit(block, NUMERODELASORTIE)` lorsque le numéro de sortie est précisé), puis libéré avec `release(block)`. Pour un objet multicanal, cette opération doit être réalisée pour chaque bloc utilisé.

Squelette :

```
void AudioEffectGain::update(void) {
  // Récupérer un bloc MODIFIABLE sur l'entrée 0.
  audio_block_t *block = receiveWritable(0);

  // TODO : vérifier que le block n'est pas nul, sinon sortir prématurément de la fonction

  // TODO : si on a activé le bypass -> on transmet tel quel, puis on libère et on sort.

  // Choisir le gain effectif (0 si mute).
  float g = muted_ ? 0.0f : gain_;

  // Traitement échantillon par échantillon.
  for (int i = 0; i < AUDIO_BLOCK_SAMPLES; i++) {
    // TODO : récupérer un échantillon dans block->data, le caster en float, puis le multiplier par le gain

    // TODO : SATURER y dans l'intervalle [-32768, +32767]

    // TODO : remplacer dans le bloc l'échantillon modifié casté en int16
  }

  // TODO : transmettre le bloc puis libérer le bloc
}
```

2.1 Types numériques et représentation des échantillons

Les échantillons audio de la bibliothèque Audio de Teensy sont stockés dans `block->data` sous la forme d'entiers signés sur 16 bits (`int16_t`). Un échantillon peut donc prendre une valeur comprise entre $-2^{15} = -32768$ et $2^{15} - 1 = 32767$.

Même si les échantillons sont stockés en `int16_t`, il est souvent préférable d'effectuer les calculs dans un type flottant (`float`). Par exemple, pour appliquer un gain :

$$y = g\,x$$

on peut écrire :

```
float y = (float)block->data[i] * gain;
```

La conversion vers `float` permet notamment d'éviter les problèmes liés aux calculs entiers. Avec des entiers, une division ou une multiplication peut entraîner une perte de précision ou un dépassement de capacité.

Sur le Teensy 4.1, basé sur un processeur Cortex-M7 disposant d'une unité de calcul flottant, les calculs en `float` sont par ailleurs suffisamment rapides pour être utilisés couramment dans le traitement audio temps réel.

À la fin du traitement, le résultat doit être reconverti en `int16_t` afin d'être stocké dans le bloc audio :

```
block->data[i] = (int16_t)y;
```

Il faut cependant faire attention aux valeurs qui dépassent l'intervalle représentable par un entier signé sur 16 bits :

$$-32768 \le y \le 32767.$$

Par exemple, si un gain supérieur à 1 est appliqué à un signal suffisamment fort, on peut obtenir :

$$y = 40000.$$

Cette valeur ne peut pas être représentée par un `int16_t`. Une conversion directe peut alors produire une valeur incorrecte, et non simplement la valeur maximale 32767.

On effectue donc généralement une **saturation** avant la conversion :

$$y_{\text{sat}} = \begin{cases} 32767 & \text{si } y > 32767, \\ -32768 & \text{si } y < -32768, \\ y & \text{sinon.} \end{cases}$$

La saturation évite ainsi qu'un dépassement de l'intervalle représentable ne se transforme en une valeur complètement différente lors de la conversion vers `int16_t`. En contrepartie, lorsqu'une saturation se produit, l'information située au-delà de la pleine échelle est perdue : en audio, cela correspond à un écrêtage (*clipping*).

Note – attributs et variables locales. Les paramètres de l'objet utilisés dans `update()` peuvent être modifiés depuis le programme principal, alors que `update()` est exécutée par le moteur audio, dans un contexte indépendant. Lorsqu'un attribut peut ainsi être modifié de manière asynchrone, il peut être déclaré `volatile`. Ce mot-clé indique au compilateur que la valeur de l'attribut peut changer

en dehors du flot d'exécution courant et que les accès à cette variable doivent donc être réellement effectués en mémoire.

Il est néanmoins préférable de ne pas accéder directement à cet attribut à chaque itération de la boucle de traitement. On peut effectuer une copie locale au début de `update()` :

```
float g = gain_;

for (int i = 0; i < AUDIO_BLOCK_SAMPLES; i++) {
  float y = (float)block->data[i] * g;
  // ...
}
```

La variable locale `g` constitue alors une photographie du paramètre au début du traitement du bloc. Même si `gain_` est modifié par le programme principal pendant l'exécution de la boucle, tous les échantillons du bloc sont traités avec la même valeur de gain.

Cette pratique présente également un intérêt pour les performances : une variable locale peut être conservée dans un registre du processeur pendant la boucle, évitant des accès répétés à la mémoire. Le compilateur est cependant souvent capable de réaliser lui-même cette optimisation lorsque cela est possible.

Ainsi, on peut distinguer :

- l'attribut `gain_` : paramètre de l'objet, potentiellement partagé avec le programme principal ;
- la variable locale `g` : copie utilisée pendant le traitement d'un bloc ;
- la boucle : tous les échantillons du bloc utilisent la même valeur de `g`.

Avec tout ce qui a été vu ci-dessus, nous arrivons au code suivant :

```
void AudioEffectGain::update(void) {
  audio_block_t *block = receiveWritable(0);

  if (!block) return;

  float g = gain_;

  for (int i = 0; i < AUDIO_BLOCK_SAMPLES; i++) {
    float y = (float)block->data[i] * g;

    block->data[i] = (int16_t)y;
  }

  transmit(block);
  release(block);
}
```

Activité 1

Dans `effect_gain.cpp`, complétez le squelette du code avec ce qui a été vu précédemment.

Attention

Sans saturation, un dépassement `int16` produit un *wrap-around* ($+32767 \to -32768$) très audible (clics). Oubli de `release()` sur un chemin ⇒ le son se fige après quelques secondes.

3 Patch

Une fois un nouvel effet créé, il faut l'intégrer dans le patch. Ici, si vous souhaitez écouter de la musique depuis votre PC plutôt que d'utiliser une entrée de la carte, vous pouvez remplacer `AudioInputI2S` par `AudioInputUSB` (le réglage *USB Type* est déjà celui utilisé pour le MIDI).

`AudioInputUSB` ⟶ `AudioEffectGain` ⟶ `AudioOutputI2S`

Attention

Les entrées et sorties I2S/USB sont **stéréo** (canaux 0 et 1), alors que votre objet ne possède qu'une entrée et une sortie. Les `AudioConnection` doivent de plus être déclarées en variables **globales**, et non dans `setup()`.

Activité 2

Dans `TP2_gain.ino`, adapter le patch avec des `AudioConnection` pour insérer votre effet gain entre l'entrée (USB ou I2S) et la sortie casque, en laissant la **voie droite en direct** de l'entrée vers la sortie. Écouter le résultat : qu'entendez-vous dans chaque oreille, et pourquoi ?

4 Budget temps réel

Activité 3

Comparer `AudioProcessorUsage()` avec et sans l'objet gain. Commenter le surcoût du traitement échantillon par échantillon.

5 Bypass et mute

Dans `effect_gain.h`, vous pouvez voir qu'il existe deux attributs `bypass_` et `muted_` pour gérer respectivement le bypass (gain unitaire) et le mute (gain nul). Ce sont des `volatile bool` : ils sont écrits depuis `loop()` à la réception des CC 80 et 81, et lus dans `update()` — c'est précisément la situation décrite en début de sujet.

Activité 4

Implémenter ces deux comportements dans `update()`, puis répondre :

1. Le bypass peut s'écrire de deux façons : une *sortie anticipée* avant la boucle, ou un simple `g = 1.0f`. Les deux s'entendent-elles différemment ? Laquelle coûte le moins de temps de

calcul, et pourquoi ?
2. Que doit-il se passer si le bypass *et* le mute sont activés simultanément ? Votre code impose un choix : lequel, et où est-il inscrit ?
3. À l'écoute, qu'entendez-vous au moment précis où vous basculez le mute alors qu'un signal est présent ? Comment l'expliquez-vous ?

6 Livrables

1. Objet `AudioEffectGain` fonctionnel, inséré dans le graphe, piloté au pupitre MIDI ;
2. Mute *et* bypass opérationnels et distincts ;
3. Relevé CPU avec/sans l'objet + une phrase d'explication du surcoût.



Remarque *Pour aller plus loin*

Rendre l'objet **stéréo** (2 entrées / 2 sorties) ; ramper le gain (`gainCourant += (gainCible - gainCourant) * 0.01f;`) pour supprimer le clic.

Pour du stéréo, il faut gérer les blocs droite et gauche séparément. Voici un squelette minimal :

```
audio_block_t *left  = receiveWritable(0);
audio_block_t *right = receiveWritable(1);

if (!left || !right) {
    if (left)  release(left);
    if (right) release(right);
    return;
}

// traitement...

transmit(left, 0);
transmit(right, 1);

release(left);
release(right);
```